

Seminar

Effective classes. Design Patterns.

1. Const correctness.

- A non-const method **cannot** be called on a **const** object or a const reference.
- Every method that does **NOT** modify the object must be declared **const**.
- Examples:

```
class Person {
public:
    const char* GetName() const;

private:
    const char* name;
};

void printPerson(const Person& p) {
    p.GetName(); // Compile error if GetName() is not const
}
```

2. Initializer list – order and const members.

- Used for initializing const members or any member that needs to be initialized before the body of the constructor.
- Separated by commas, used like a constructor for the members.
- Other constructors can be called using the initializer list.
- For **const** members and **references**, the initializer list is **not optional** – it is the **only** way to initialize them.

- Example:

```
class Config {
public:
    Config(int r, int& x) : maxRetries(r), ref(x) {}
    // Must use the initializer list for both!
private:
    const int maxRetries;
    int& ref;
};
```

- Members are initialized in **declaration order**, not in the order they appear in the initializer list.

- Example:

```
class Bad {
public:
    Bad(int x) : a(x), b(a) {}
    // `b` uses `a`, but `b` is initialized before `a`!
private:
    int b; // declared first → initialized first
    int a; // declared second
};
```

- **Rule:** Write the initializer list in the **same order** as the member declarations.

3. Why does operator=(...) return a reference?

- Returns a reference to ***this** to allow **chained assignment**:
`a = b = c; // equivalent to: a = (b = c);`
- If operator= returned **void**, chained assignment would not compile.
- If it returned by **value** (`Class` instead of `Class&`), an unnecessary copy would be made and chaining would still behave unexpectedly.
- Standard implementation:

```
Person& Person::operator=(const Person& other) {  
    if (this == &other) return *this;  
    // ... copy members ...  
    return *this;  
}
```

4. The explicit keyword.

- Single-argument constructors act as **implicit conversion constructors** by default – the compiler can call them automatically.
- Marking a constructor **explicit** prevents the compiler from using it for implicit conversions.
- Without explicit - dangerous implicit conversion:

```
class MyString {  
public:  
    MyString(int size); // implicit: MyString s = 42; compiles!  
};
```

- With explicit - safe:

```
class MyString {  
public:  
    explicit MyString(int size);  
};  
// MyString s = 42; // Won't compile  
MyString s(42); // Explicit construction still works
```

Rule of thumb: mark every single-argument constructor **explicit** unless you intentionally want implicit conversion.

5. Static data members.

- **Static** data members belong to the **class**, not to any instance. They exist even when zero objects have been created.
- They are **declared** in the header but must be **defined** (and optionally initialized) in the **.cpp** file.
- Static **methods** have no **this** pointer - they can only access static members.

— *Person.h* —

```
class Person {  
public:  
    static int GetCount() { return count; }  
private:  
    static int count; // Declaration only  
};
```

— *Person.cpp* —

```
int Person::count = 0; // Definition - must be in the .cpp
```

6. Exception Safety Guarantee.

- When an exception occurs during an operation, there are three levels of guarantee:
 - **None** – anything can happen; the program or object may be left in a corrupt or undefined state.
 - **Basic** – the program and object will be in a **valid** state (though it may be changed or empty). No resources are leaked.
 - **Strong** – the program and object will be in the **exact same state** as before the operation was attempted.
- Example – strong guarantee with copy-and-swap:

```
// Copy made before anything changes
Person& Person::operator=(Person other) {
    swap(*this, other); // swap must not throw exceptions
    return *this;
    // Old data freed when 'other' goes out of scope
}
```

7. Encapsulation and getter return types.

- A **struct** gives direct public access to its members. A **class** encapsulates state and exposes it through methods, allowing invariants to be enforced.
- struct – no protection:

```
struct ComposedHero {
    String name;
    String lastName;
    short health = 0;
};
```
- class – with encapsulation:

```
class ComposedHero {
public:
    ComposedHero(const String& name, const String& last, short hp);
    const String& GetName() const;
    const String& GetLastName() const;
    short GetHealth() const;
    void SetHealth(short hp);

private:
    String name;
    String lastName;
    short health = 0;
};
```
- Why `const String& GetName() const` ?
 - **No ownership ambiguity** - who owns the `char*`? With `const String&` the lifetime is clear.
 - **Reference avoids copy** - returning by `const` reference is zero-cost and if a copy is needed - it's easily achievable:

```
const String& ref = h.GetName(); // no copy
String copy = h.GetName(); // copy
```

8. Nested classes.

- A class can be defined **inside** another class. The inner class is **scoped** to the outer one and referred to as `Outer::Inner` from outside.
- Use nested classes for implementation details that **belong to one class** and should not pollute the global namespace.

- Example:

```
class LinkedList {
private:
    struct Node { // Node only makes sense inside LinkedList
        int data;
        Node* next;
    };

    Node* head;
};

// Referred to as:
LinkedList::Node* ptr;
```

9. Friend classes and friend functions.

- Classes and functions can be declared as **friends** to a given class.
- Friends of the class can access **all private** data members and methods.
- Friendship is **granted by the class** itself - it cannot be taken. It is **not inherited** and not transitive.
- Example - an iterator needs direct access to a LinkedList's private head pointer. A setter/getter cannot solve this; the two classes are one logical unit:

```
class LinkedList {
    friend class LinkedListIterator; // Grants access
private:
    Node* head;
};

class LinkedListIterator {
public:
    LinkedListIterator(LinkedList& list) { current = list.head; }
                                     // direct access to private head
    int Get() { return current->data; }
    void Next() { current = current->next; }

private:
    LinkedList::Node* current;
};
```

10. Controlling object creation.

- By making the constructor **private** and providing a static **factory method**, we can guarantee **exactly one instance** of a class ever exists.
- The copy constructor and copy-assignment operator are **deleted** so the single instance cannot be duplicated.

- Implementation:
Singleton **Design Pattern**

```
class Singleton {
public:
    static Singleton& GetInstance() {
        // created once, destroyed at program end
        static Singleton instance;
        return instance;
    }
    Singleton(const Singleton&) = delete; // no copying
    Singleton& operator=(const Singleton&) = delete; // no op=

private:
    Singleton() = default; // private constructor
};
```

11. Fluent Interface / Method Chaining.

- Operations that return ***this** by reference allow calls to be **chained**.
- Example:

```
class QueryBuilder {
public:
    QueryBuilder& Select(const String& cols) {
        query += "SELECT " + cols + " ";
        return *this;
    }
    QueryBuilder& From(const String& table) {
        query += "FROM " + table + " ";
        return *this;
    }
    QueryBuilder& Where(const String& cond) {
        query += "WHERE " + cond + " ";
        return *this;
    }

    // Question: Why doesn't build() return const String& ?
    String Build() const { return query; }
public:
    String query;
};
```

Usage reads almost like English:

```
String query = QueryBuilder()
    .Select("name, age")
    .From("users")
    .Where("age > 18")
    .Build();
```

- Returning ***this by reference** is the same mechanism that makes operator<< chainable on streams.
* More on this in the next two seminars *

```
std::cout << "An example: " << str << "\n";
```